

Module 4: Advanced Topics (Modular Extensions) - Complete Study Notes

Welcome to **Module 4** of our Java File Handling course! Congratulations on completing the basics—this final module ties everything together with advanced techniques. Unlike previous modules, this one is **extension-based**: Pick and mix topics like NIO (New I/O), serialization, error handling, and multi-threading to add depth. These build real-world skills for performance, object persistence, and robustness.

By the end of this module, you'll:

- Use modern NIO for non-blocking I/O.
- Serialize/deserialize Java objects to binary files.
- Implement robust error handling and best practices.
- Handle concurrent file access safely.

Key Concepts:

- **NIO (`java.nio`)**: Buffer + Channel model for faster, scalable I/O (Java 7+).
- **Serialization**: Save/load objects as binary streams (implements `Serializable`).
- **Advanced Error Handling**: Custom exceptions, logging.
- **Multi-Threading**: `RandomAccessFile` for shared access.
- Focus: Choose 1-2 extensions; code is modular.

Prerequisites: Modules 1-3. Java 8+. For NIO/serialization, import `java.nio.*` and `java.io.*`.

To visualize NIO's core components (Channels, Buffers, Paths), here's a diagram:

1. Core Concepts: Advanced File Handling Overview

Traditional I/O (streams) is sequential and blocking; advanced tools add speed, flexibility:

- **NIO**: Path-based (replaces `File`), Channels (bidirectional), Buffers (memory pools).
- **Serialization**: `ObjectOutputStream` writes objects as binary; reverse for input.
- **Error Handling**: Beyond `IOException` —use try-catch-finally or custom checks.
- **Concurrency**: Locks or atomic ops for multi-thread safety.

Import Note: Mix packages— `java.nio.file.*` for NIO, `java.beans.*` optional for XML serialization.

2. Step-by-Step Code Examples with Comments

Each extension is a self-contained subsection with runnable examples. Start with NIO (most impactful).

Extension 1: Error Handling & Best Practices

Enhance all prior code with logging, validation, and `Files` one-liners (NIO hybrid).

Sample Code: `RobustFileCopier.java` (Combines Modules 2/3 with checks)

```

import java.io.*;
import java.nio.file.*; // For Files utility
import java.util.logging.Logger; // Built-in logging

public class RobustFileCopier {
    private static final Logger LOGGER = Logger.getLogger(RobustFileCopier.class.getName());

    /**
     * Copies file with full error handling and validation.
     * @param source Source path
     * @param dest Destination path
     * @return true if successful
     */
    public static boolean copyWithChecks(String source, String dest) {
        Path sourcePath = Paths.get(source); // NIO Path for easy checks
        Path destPath = Paths.get(dest);

        // Pre-checks: Best practice—validate before I/O
        if (!Files.exists(sourcePath)) {
            LOGGER.warning("Source not found: " + source);
            return false;
        }
        if (Files.isDirectory(sourcePath)) {
            LOGGER.warning("Source is directory: " + source);
            return false;
        }
        try {
            // Use NIO Files.copy()—simple, handles text/binary automatically
            Files.copy(sourcePath, destPath, StandardCopyOption.REPLACE_EXISTING);
            LOGGER.info("Copied " + source + " to " + dest);
            return true;
        } catch (AccessDeniedException e) {
            // Specific catch: Permission issues
            LOGGER.severe("Access denied: " + e.getMessage());
        } catch (IOException e) {
            // General I/O
            LOGGER.severe("IO error: " + e.getMessage());
            e.printStackTrace();
        }
        return false;
    }

    public static void main(String[] args) {

```

```
// Test: Copy from Module 3
copyWithChecks("data/input.jpg", "data/output_safe.jpg");
}
}
```

Output: Logs success/failure. **Why?** Prevents crashes; use `standardCopyOption` for append/overwrite.

Best Practice Adder: Always log with `java.util.logging` —configurable levels (INFO, WARNING, SEVERE).

Extension 2: NIO for Modern I/O

`Path` + `Files` for metadata; `Channels` + `ByteBuffer` for direct I/O (faster than streams).

Sample Code: NioPathBasics.java (Replaces Module 1 File)

```

import java.nio.file.*; // Path, Files
import java.io.IOException;

public class NioPathBasics {
    public static void main(String[] args) {
        Path path = Paths.get("data", "nio_example.txt"); // Builder: like File("data/nio_example.txt")

        try {
            // Check properties (like File.exists())
            System.out.println("Exists: " + Files.exists(path));
            System.out.println("Size: " + Files.size(path) + " bytes");

            // Create dirs/file
            Files.createDirectories(path.getParent()); // Like mkdirs()
            Files.createFile(path); // Empty file

            // List contents (like listFiles())
            try (DirectoryStream<Path> stream = Files.newDirectoryStream(path.getParent())) {
                System.out.println("Parent contents:");
                for (Path p : stream) {
                    System.out.println("- " + p.getFileName());
                }
            }
        } catch (IOException e) {
            System.err.println("NIO error: " + e.getMessage());
        }
    }
}

```

Output: Similar to Module 1, but more concise. **Pro:** Path is immutable/safe.

Channel/Buffer Read/Write (Replaces Module 3 Binary):

Sample Code: NioChannelCopy.java

```

import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.*;

public class NioChannelCopy {
    public static void main(String[] args) {
        Path source = Paths.get("data/input.jpg");
        Path dest = Paths.get("data/output_nio.jpg");

        try (FileChannel inChannel = FileChannel.open(source, StandardOpenOption.READ);
            FileChannel outChannel = FileChannel.open(dest, StandardOpenOption.CREATE, StandardOpenOption.WRITE)) {

            // Allocate buffer (direct for OS speed: ByteBuffer.allocateDirect(4096))
            ByteBuffer buffer = ByteBuffer.allocate(4096);

            // Loop: Read to buffer, flip (ready for write), write, clear for next
            while (inChannel.read(buffer) != -1) { // read() returns -1 at EOF
                buffer.flip(); // Limit to position, position=0—now writable
                outChannel.write(buffer); // Write what's in buffer
                buffer.clear(); // Reset for reading: position=0, limit=capacity
            }
            System.out.println("NIO copy complete!");
        } catch (IOException e) {
            System.err.println("Channel error: " + e.getMessage());
        }
    }
}

```

Why NIO? Non-blocking possible (advanced); 2x faster for large files.

Extension 3: Serialization (Binary Objects)

Save/load Java objects to binary files—great for state persistence.

Sample Code: SerializableExample.java

```

import java.io.*;

// Class must implement Serializable for magic
class Person implements Serializable { // Serializable marker interface—no methods
    private static final long serialVersionUID = 1L; // Version for changes
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getters (no setters needed for demo)
    public String getName() { return name; }
    public int getAge() { return age; }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}

public class SerializableExample {
    public static void main(String[] args) {
        String filename = "data/person.ser"; // .ser convention for serialized

        // Write (serialize)
        Person p = new Person("Alice", 30);
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename))) {
            oos.writeObject(p); // Writes object graph as binary
            System.out.println("Serialized: " + p);
        } catch (IOException e) {
            System.err.println("Serialize error: " + e.getMessage());
        }

        // Read (deserialize)
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filename))) {
            Person readP = (Person) ois.readObject(); // Cast to type
            System.out.println("Deserialized: " + readP);
        } catch (IOException | ClassNotFoundException e) { // ClassNotFoundException if class changed
            System.err.println("Deserialize error: " + e.getMessage());
        }
    }
}

```

```
}  
}
```

Output:

```
Serialized: Person{name='Alice', age=30}  
Deserialized: Person{name='Alice', age=30}
```

Caveat: Transients skipped (e.g., `transient` fields); version UID prevents mismatches.

Extension 4: Multi-Threading

Use `RandomAccessFile` for concurrent read/write at offsets (e.g., append without full reopen).

Sample Code: `ConcurrentAppender.java` (Thread-safe append)


```

import java.io.*;

public class ConcurrentAppender {
    /**
     * Appends to file from multiple threads—use seek for position.
     */
    public static void appendSafely(String filename, String message) {
        try (RandomAccessFile raf = new RandomAccessFile(filename, "rw")) { // rw = read/write
            // Seek to end for append
            raf.seek(raf.length()); // Position at EOF
            raf.writeBytes(message + "\n"); // Write as bytes
            System.out.println("Appended: " + message);
        } catch (IOException e) {
            System.err.println("Append error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        String filename = "data/concurrent_log.txt";

        // Simulate threads: Run 3 "threads" sequentially (use ExecutorService for real)
        appendSafely(filename, "Thread 1: Hello");
        appendSafely(filename, "Thread 2: World");
        appendSafely(filename, "Thread 3: Concurrent!");
    }
}

```

For True Concurrency: Wrap in `synchronized` or use `ReentrantLock`. **Pro:** Efficient for logs.

3. Practice: Building Reusable Methods

Combine extensions in a "AdvancedFileManager".

Classwork 1: Integrated Manager

Handles copy + serialize + NIO checks.

Complete Class: `AdvancedFileManager.java`

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class AdvancedFileManager {
    /**
     * Copies file using NIO, logs errors, returns metadata.
     */
    public static Map<String, Object> smartCopy(String source, String dest) {
        Map<String, Object> result = new HashMap<>();
        Path srcPath = Paths.get(source);
        Path dstPath = Paths.get(dest);

        try {
            if (!Files.exists(srcPath)) {
                result.put("success", false);
                result.put("error", "Source not found");
                return result;
            }
            Files.copy(srcPath, dstPath, StandardCopyOption.REPLACE_EXISTING);
            result.put("success", true);
            result.put("size", Files.size(dstPath));
        } catch (IOException e) {
            result.put("success", false);
            result.put("error", e.getMessage());
        }
        return result;
    }

    /**
     * Serializes a list of objects.
     */
    public static void serializeList(String filename, List<?> objects) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename))) {
            oos.writeObject(objects); // Writes entire list
            System.out.println("Serialized list of " + objects.size() + " objects");
        } catch (IOException e) {
            System.err.println("List serialize error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        // Test copy
    }
}
```

```
Map<String, Object> copyResult = smartCopy("data/sample.txt", "data/copy_nio.txt");
System.out.println("Copy result: " + copyResult);

// Test serialize
List<Person> people = Arrays.asList(new Person("Bob", 25), new Person("Carol", 35));
serializeList("data/people.ser", people);
}
}
```

Output: Copy metadata map; serialized list. **Why Integrated?** Real apps mix features.

Depth Adder: NIO Watcher (File Change Detection)

Enhanced Class: FileWatcher.java

```

import java.nio.file.*;

public class FileWatcher {
    /**
     * Watches for file changes (advanced NIO).
     */
    public static void watchPath(String pathStr) {
        Path path = Paths.get(pathStr);
        try {
            WatchService watcher = FileSystems.getDefault().newWatchService();
            path.register(watcher, StandardWatchEventKinds.ENTRY_CREATE, StandardWatchEventKinds.ENTRY_DELETE, StandardWatchEventKinds.ENTRY_MODIFY);

            System.out.println("Watching " + path + "...");
            while (true) { // Infinite loop—use thread in real
                WatchKey key = watcher.take(); // Blocks until event
                for (WatchEvent<?> event : key.pollEvents()) {
                    System.out.println("Event: " + event.kind() + " on " + event.context());
                }
                key.reset(); // Re-arm
            }
        } catch (Exception e) {
            System.err.println("Watcher error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        watchPath("data"); // Touch files in data/ to see events
    }
}

```

Use: Run, modify "data/output.txt"—prints MODIFY.

4. Tasks and Exercises

Advanced challenges—pick 2+ extensions.

Task 1: NIO Migrator (Beginner - 15 mins)

- Class: `NioMigrator` .
- Method: `migrateOld(String oldFile)` —read with streams (Module 2), write with NIO `Files.write` .
- Test: Migrate text/binary, compare sizes.
- **Expected:** Use `ByteBuffer` for binary.

Task 2: Serializable Database (Intermediate - 20 mins)

- Class: `ObjectDB` .
- Methods: `save(Object obj, String key)` , `load(String key)` —use `Map<String, Object>` serialized to file.
- Test: Save Person list, load, print.
- **Challenge:** Handle version changes with `serialVersionUID`.

Task 3: Threaded Logger (Advanced - 25 mins)

- Extend Module 2 Logger: Use threads (`ExecutorService`) to append concurrently.
- Method: `logAsync(String message)` —queue and write in background.
- **Bonus:** Use NIO `AsynchronousFileChannel` for non-blocking.

Task 4: Custom Exception Handler (Debugging - 15 mins)

- Create `FileHandlingException` extends `Exception`.
- Throw in `smartCopy` for specific errors (e.g., too large >1MB).
- Test: Catch and log.

Grading Rubric (Self-Assess):

- Integrates 2+ extensions: 40%
- Commented advanced parts (e.g., `buffer.flip()`): 30%
- Handles concurrency/errors: 20%
- Efficient (NIO where possible): 10%

5. Common Pitfalls and Best Practices

Pitfall	Fix
NIO buffer not flipped	Always read->flip->write->clear cycle
Serialization security	Validate input objects; use Externalizable for control
Concurrent overwrites	Use file locks: <code>FileLock lock = raf.getChannel().lock();</code>
Large object graphs	Use transient for big fields; compress with GZIP
Mixing old/new I/O	Prefer NIO for new code—streams for legacy

Pro Tip: For web/apps, use libraries like Apache Commons IO. Profile performance with JVisualVM.

Resources:

- Oracle Docs: [java.nio.file](#), [Serialization](#)
- Practice: Migrate Module 3 copier to NIO, benchmark time.

Course Complete! You've got a full toolkit—build a project like a file sync tool. Questions or expansions? Let's discuss!